Assignment 10

Aim - Implement a program to count number of leaf nodes, biggest and smallest number and height of a binary search tree.

Theory

A binary search tree is a tree in which the left node has the smaller number and right node has the larger number than its root.

A leaf node is a node where the left and right are both null, the tree ends at them.

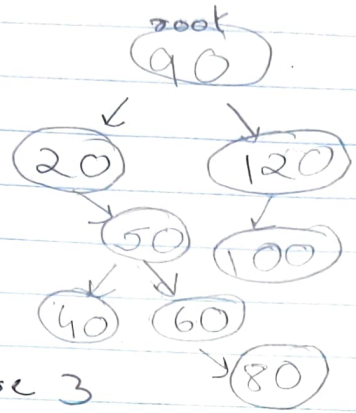
The smallest element is the leftmost leaf node because smaller values always on the left

The largeot element is the rightmost leaf node because largest values are alway on the right

Height of a atree is the number of edges or levels from root to deepest leaf.

THADOMAL SHAHANI
TSEC
ENGINEERING COLLEGE

e.g.



Sample data
90, 20, 50, 120, 100, 40, 60, 80

smallest → leftmost → 20
largest → Rightmost → 120
height → 3 (3 branches + root)

leaf nodes are 3
as they are at the ends :- 40, 80, 100

Conclusion

Binary search tree was successfully implemented. Functions were created to count leaf nodes, determine the ~~sma~~ smallest and largest elements, ~~calc~~ and calculate the height of the tree. The result verified that BST properties allow efficient searching and traversal operations.

```
#include <stdio.h>
#include <stdlib.h>

typedef struct node {
   int data;
   struct node *left;
   struct node *right;
} Node;

Node* createNode(int value) {
   Node* newNode = (Node*)malloc(sizeof(Node));
   newNode->data = value;
   newNode->left = newNode->right = NULL;
   return newNode;
}

Node* insert(Node* root, int value) {
   if (root == NULL)
      return createNode(value);
   if (value < root->data)
      root->left = insert(root->left, value);
   else
      root->right = insert(root->right, value);
   return root;
}

int countLeafNodes(Node* root) {
   if (root == NULL)
      return 0;
   if (root->left == NULL && root->right == NULL)
      return 1;
```

```
    return countLeafNodes(root->left) + countLeafNodes(root->right);
}

int findMin(Node* root) {
    if (root == NULL) {
        printf("Tree is empty\n");
        return -1;
    }
    while (root->left != NULL)
        root = root->left;
    return root->data;
}

int findMax(Node* root) {
    if (root == NULL) {
        printf("Tree is empty\n");
        return -1;
    }
    while (root->right != NULL)
        root = root->right;
    return root->data;
}

int height(Node* root) {
    if (root == NULL)
        return 0;
    int leftHeight = height(root->left);
    int rightHeight = height(root->right);
    return (leftHeight > rightHeight ? leftHeight : rightHeight) + 1;
}
```

```
void main() {
    Node* root = NULL;
    int values[] = {90, 20, 50, 120, 100, 40, 60, 80};
    int n = 7;
    for (int i = 0; i < n; i++)
        root = insert(root, values[i]);
    printf("Number of leaf nodes: %d\n", countLeafNodes(root));
    printf("Smallest value: %d\n", findMin(root));
    printf("Largest value: %d\n", findMax(root));
    printf("Height of tree: %d\n", height(root));
}
```

```
Number of leaf nodes: 3
Smallest value: 20
Largest value: 120
Height of tree: 4
```